Formal Verification of CategoricalCellularAutomata.jl

Simon Frost

2026-11-03

Contents

Formal verification of CategoricalCellularAutomata.jl 1

Configurations, shifts and pullbacks 1

Cellular automata over a group 3

Generalised cellular automata and pullbacks 4

Gluing spaces 5

Formal verification of CategoricalCellularAutomata.jl

This library proves, in Lean 4 with Mathlib, the laws that `CategoricalCellularAutomata.jl` relies on and that its test suite checks numerically. The definitions are the mathematical objects the Julia types implement, over any group (finite or not), with any alphabet; the Julia package works with finite groups and alphabets, so its computations are instances.

Lean	Julia	statement
`Config G A`	`Configuration`	a configuration `G → A`
`shift g x`	`shift(x, g)`	`(g · x)(h) = x(h g)`, a left action (`shift_one`, `shift_shift`)
`pullback φ x`	`pullback(φ, X)`	`x ∘ φ`, contravariantly functorial (`pullback_id`, `pullback_comp`)
`ca S μ`	`CellularAutomaton(G, S, μ)`	`τ(x)(g) = μ((x(s g))_{s ∈ S})`
`ca_shift`	`is_equivariant`	every cellular automaton commutes with the shifts
`ca_eq_iff`	`σ == τ`	two automata are equal when they agree at the identity
`ca_id`, `ca_comp`	`id`, `compose`	the identity, and composites with memory set `T S`
`ca_pair`	`pair`	pairing over the disjoint union of memory sets
`gca φ S μ`	`GeneralizedCA(φ, S, μ)`	`T(x)(h) = μ((x(s φ(h)))_{s ∈ S})`
`gca_eq_pullback_ca`	`factorize`	`T = φ* ∘ τ`
`pullback_ca`	`pullback`, `SpaceMorphism`	pullback commutes with local rules: `φ* ∘ τ_{φ(S)} = τ_S ∘ φ*`
`gca_comp`	`compose`	generalised automata compose along `φ ∘ ψ`
`gluedConfigEquiv`, `exists_unique_glued_config`	`glue`	configurations of a glued space are the compatible families on its patches (a limit)
`step_restrict`, `step_eq_of_cover`	`glue`, `apply_local`	the landscape's step on a patch's interior is the patch's step

Every theorem in the default target is free of `sorry`; `Audit.lean` prints the axioms each main theorem uses (only `propext`, `Classical.choice` and `Quot.sound` are expected).

Configurations, shifts and pullbacks

```
import Mathlib.Algebra.Group.Hom.Defs
```

A configuration over a group `G` with values in an alphabet `A` is a function `x : G → A` (`Configuration` in `CategoricalCellularAutomata.jl`). The group acts on configurations on the left by `(g · x)(h) = x (h * g)` (`shift(x, g)`), and a homomorphism `φ : H →* G` pulls a configuration over `G` back to one over `H` by precomposition (`pullback(φ, X)`).

```
namespace CategoricalCellularAutomataProofs

/-- A configuration over `G` with values in `A`: the state of a cellular automaton. -/
abbrev Config (G : Type*) (A : Type*) := G → A

variable {G H K A : Type*}

section Shift

variable [Monoid G]

/-- The shift `g · x`, with `(g · x)(h) = x (h * g)` (`shift(x, g)`). -/
def shift (g : G) (x : Config G A) : Config G A := fun h ⇒ x (h * g)

@[simp] theorem shift_apply (g h : G) (x : Config G A) : shift g x h = x (h * g) := rfl

/-- The identity acts trivially. -/
@[simp] theorem shift_one (x : Config G A) : shift 1 x = x := by
  funext h; simp [shift]

/-- Shifting is a left action: `g · (g' · x) = (g g') · x`. -/
theorem shift_shift (g g' : G) (x : Config G A) : shift g (shift g' x) = shift (g * g') x := by
  funext h; simp [shift, mul_assoc]

end Shift

section Pullback

/-- The pullback `φ* x = x ∘ φ` of a configuration along a map of groups (`pullback(φ, X)`). -/
def pullback [MulOneClass H] [MulOneClass G] (φ : H →* G) (x : Config G A) : Config H A :=
  fun h ⇒ x (φ h)

@[simp] theorem pullback_apply [MulOneClass H] [MulOneClass G] (φ : H →* G) (x : Config G A)
    (h : H) : pullback φ x h = x (φ h) := rfl

/-- Pulling back along the identity changes nothing. -/
@[simp] theorem pullback_id [MulOneClass G] (x : Config G A) :
    pullback (MonoidHom.id G) x = x := rfl

/-- Pullback is contravariantly functorial: `(φ ∘ ψ)* = ψ* ∘ φ*`. -/
theorem pullback_comp [MulOneClass K] [MulOneClass H] [MulOneClass G] (φ : H →* G)
    (ψ : K →* H) (x : Config G A) :
    pullback (φ.comp ψ) x = pullback ψ (pullback φ x) := rfl

/-- Pullback intertwines the shifts: `φ* (φ(h) · x) = h · φ* x`. -/
theorem pullback_shift [Monoid H] [Monoid G] (φ : H →* G) (h : H) (x : Config G A) :
    pullback φ (shift (φ h) x) = shift h (pullback φ x) := by
  funext k; simp [pullback, shift, map_mul]

end Pullback

end CategoricalCellularAutomataProofs
```

Cellular automata over a group

```
import CategoricalCellularAutomataProofs.Configurations
```

A cellular automaton over `G` with memory set `S` and local rule `μ` maps a configuration `x` to `τ(x)(g) = μ((x(s g))_{s ∈ S}) (CellularAutomaton(G, S, μ))`. The memory set is an ordered list of group elements, here a family `S : I → G` indexed by any type `I` (in Julia, `I = Fin (length S)`), and the local rule reads the values at `S i * g` in that order.

This file proves that every cellular automaton commutes with the shifts, that a cellular automaton is determined by its values at the identity (which is how `σ == τ` decides equality), that the identity map is a cellular automaton, that the composite of two is a cellular automaton over the product of their memory sets (`compose`), and that pairing two cellular automata is one cellular automaton over the disjoint union of their memory sets (`pair`).

```
namespace CategoricalCellularAutomataProofs

variable {G A B C : Type*} {I J : Type*}

/-- The cellular automaton with memory set `S` and local rule `μ`:
`τ(x)(g) = μ (fun i ⇒ x (S i * g))`. -/
def ca [Mul G] (S : I → G) (μ : (I → A) → B) (x : Config G A) : Config G B :=
  fun g ⇒ μ (fun i ⇒ x (S i * g))

@[simp] theorem ca_apply [Mul G] (S : I → G) (μ : (I → A) → B) (x : Config G A) (g : G) :
    ca S μ x g = μ (fun i ⇒ x (S i * g)) := rfl

section Monoid

variable [Monoid G]

/-- **Equivariance.** A cellular automaton commutes with every shift: `τ (g · x) = g · τ x`. -/
theorem ca_shift (S : I → G) (μ : (I → A) → B) (g : G) (x : Config G A) :
    ca S μ (shift g x) = shift g (ca S μ x) := by
  funext h; simp [ca, shift, mul_assoc]

/-- A shift-equivariant map is determined by its values at the identity:
`τ(x)(g) = τ(g · x)(1)`. -/
theorem apply_eq_apply_one_shift (τ : Config G A → Config G B)
    (hτ : ∀ g x, τ (shift g x) = shift g (τ x)) (x : Config G A) (g : G) :
    τ x g = τ (shift g x) 1 := by
  rw [hτ]; simp

/-- **Equality at the identity.** Two shift-equivariant maps agree when they agree at the
identity element on every configuration; this is how `σ == τ` compares cellular automata. -/
theorem eq_of_apply_one (σ τ : Config G A → Config G B)
    (hσ : ∀ g x, σ (shift g x) = shift g (σ x)) (hτ : ∀ g x, τ (shift g x) = shift g (τ x))
    (h : ∀ x, σ x 1 = τ x 1) : σ = τ := by
  funext x g
  rw [apply_eq_apply_one_shift σ hσ, apply_eq_apply_one_shift τ hτ, h]

/-- Two cellular automata are equal as maps exactly when they agree at the identity. -/
theorem ca_eq_iff {S : I → G} {T : J → G} {μ : (I → A) → B} {ν : (J → A) → B} :
    ca S μ = ca T ν ↔ ∀ x, ca S μ x 1 = ca T ν x 1 := by
  constructor
  · intro h x; rw [h]
  · intro h
    exact eq_of_apply_one _ _ (fun g x ⇒ ca_shift S μ g x) (fun g x ⇒ ca_shift T ν g x) h

/-- The identity map is the cellular automaton with memory set `{1}` that reads its one
value. -/
theorem ca_id (x : Config G A) : ca (fun _ : Unit ⇒ (1 : G)) (fun p ⇒ p ()) x = x := by
  funext g; simp [ca]

/-- **Composition.** Running `τ = ca T ν` and then `σ = ca S μ` is the cellular automaton
whose memory set is the family of products `T j * S i` (the set `T S` of `compose`) and whose
local rule applies `ν` to each block and `μ` to the results. -/
```

```
theorem ca_comp (S : I → G) (μ : (I → B) → C) (T : J → G) (ν : (J → A) → B)
    (x : Config G A) :
    ca S μ (ca T ν x) =
      ca (fun p : I × J ⇒ T p.2 * S p.1) (fun q ⇒ μ (fun i ⇒ ν (fun j ⇒ q (i, j)))) x := by
  funext g; simp [ca, mul_assoc]

/-- The composite is again equivariant, as a cellular automaton must be. -/
theorem ca_comp_shift (S : I → G) (μ : (I → B) → C) (T : J → G) (ν : (J → A) → B) (g : G)
    (x : Config G A) :
    ca S μ (ca T ν (shift g x)) = shift g (ca S μ (ca T ν x)) := by
  rw [ca_shift, ca_shift]

/-- **Pairing.** The pair of two cellular automata over the same group is one cellular
automaton whose memory set is the disjoint union of theirs (`pair`, the union `S ∪ T`). -/
theorem ca_pair (S : I → G) (μ : (I → A) → B) (T : J → G) (ν : (J → A) → C)
    (x : Config G A) (g : G) :
    ca (Sum.elim S T) (fun p ⇒ (μ (fun i ⇒ p (Sum.inl i)), ν (fun j ⇒ p (Sum.inr j)))) x g =
      (ca S μ x g, ca T ν x g) := rfl

end Monoid

end CategoricalCellularAutomataProofs
```

Generalised cellular automata and pullbacks

```
import CategoricalCellularAutomataProofs.CellularAutomata
```

For a homomorphism `φ : H →* G`, a generalised cellular automaton maps configurations over `G` to configurations over `H` by `T(x)(h) = μ((x(s φ(h)))_{s ∈ S})` (`GeneralizedCA(φ, S, μ)`).

This file proves the facts the Julia package relies on:

- `T` is the pullback along `φ` after the ordinary cellular automaton with the same memory set and rule (`factorize`);
- **pullback commutes with local rules**: pulling back the output of a cellular automaton over `G` whose memory set is the image `φ(S)` is the same as running the cellular automaton over `H` with memory set `S` on the pulled-back configuration;
- `T` intertwines the shifts along `φ`;
- two generalised automata compose to one, along the composite homomorphism, with memory set the products `S i * φ(T j)` (the set `T_φ(S)` of `compose`).

```
namespace CategoricalCellularAutomataProofs

variable {G H K A B C : Type*} {I J : Type*}

section

variable [Monoid G] [Monoid H] [Monoid K]

/-- The generalised cellular automaton along `φ` with memory set `S` and local rule `μ`. -/
def gca (φ : H →* G) (S : I → G) (μ : (I → A) → B) (x : Config G A) : Config H B :=
  fun h ⇒ μ (fun i ⇒ x (S i * φ h))

@[simp] theorem gca_apply (φ : H →* G) (S : I → G) (μ : (I → A) → B) (x : Config G A)
    (h : H) : gca φ S μ x h = μ (fun i ⇒ x (S i * φ h)) := rfl

/-- **Factorisation.** `T = φ* ∘ τ` (`factorize(T) == (φ, τ)`). -/
theorem gca_eq_pullback_ca (φ : H →* G) (S : I → G) (μ : (I → A) → B) (x : Config G A) :
    gca φ S μ x = pullback φ (ca S μ x) := rfl

/-- An ordinary cellular automaton is the generalised one over the identity homomorphism
(`GeneralizedCA(τ)`). -/
theorem gca_id (S : I → G) (μ : (I → A) → B) (x : Config G A) :
```

```
    gca (MonoidHom.id G) S μ x = ca S μ x := rfl

/-- **Pullback commutes with local rules.** For a memory set `S` in `H`, running the cellular
automaton over `G` with memory set `φ(S)` and pulling back equals pulling back and running the
cellular automaton over `H` with memory set `S`: `φ* ∘ τ_{φ(S)} = τ_S ∘ φ*`. -/
theorem pullback_ca (φ : H →* G) (S : I → H) (μ : (I → A) → B) (x : Config G A) :
    pullback φ (ca (fun i ⇒ φ (S i)) μ x) = ca S μ (pullback φ x) := by
  funext h; simp [pullback, ca, map_mul]

/-- The same, as an equality of generalised and ordinary automata:
`T_{φ, φ(S)} = τ_S ∘ φ*`. -/
theorem gca_map_eq_ca_pullback (φ : H →* G) (S : I → H) (μ : (I → A) → B)
    (x : Config G A) :
    gca φ (fun i ⇒ φ (S i)) μ x = ca S μ (pullback φ x) :=
  pullback_ca φ S μ x

/-- A generalised automaton intertwines the shifts along `φ`: `T (φ(h) · x) = h · T x`. -/
theorem gca_shift (φ : H →* G) (S : I → G) (μ : (I → A) → B) (h : H) (x : Config G A) :
    gca φ S μ (shift (φ h) x) = shift h (gca φ S μ x) := by
  funext k; simp [gca, shift, map_mul, mul_assoc]

/-- **Composition.** For `φ : H →* G` and `ψ : K →* H`, running `T = gca φ S μ` and then
`U = gca ψ T ν` is the generalised automaton along `φ ∘ ψ` whose memory set is the family of
products `S i * φ(T j)`, with the local rule applying `μ` to each block and `ν` to the
results. -/
theorem gca_comp (φ : H →* G) (S : I → G) (μ : (I → A) → B) (ψ : K →* H) (T : J → H)
    (ν : (J → B) → C) (x : Config G A) :
    gca ψ T ν (gca φ S μ x) =
      gca (φ.comp ψ) (fun p : J × I ⇒ S p.2 * φ (T p.1))
        (fun q ⇒ ν (fun j ⇒ μ (fun i ⇒ q (j, i)))) x := by
  funext k; simp [gca, map_mul, mul_assoc]

end

end CategoricalCellularAutomataProofs
```

Gluing spaces

```
import Mathlib.CategoryTheory.Limits.IsLimit
import Mathlib.CategoryTheory.Types.Basic
import Mathlib.Data.List.Basic
```

`glue` builds a landscape as the colimit of patches: each patch is a set of cells (with the slots its interior cells read), the patches share cells along their feet, and the landscape's cells are the colimit of the patches' cells. This file proves the two facts the Julia tests check numerically.

Configurations form a limit. A configuration of the landscape is a function from its cells to the alphabet, a morphism out of the colimit in `Type`. So configurations of the landscape are exactly the families of configurations of the patches that agree wherever patches share cells: the colimit of cells becomes a limit of configurations. Any colimit of cell sets will do: Catlab's colimit along an undirected wiring diagram and the package's union-find give the same one up to isomorphism.

A step restricted to a patch is the patch's step. A cell reads the cells of its slots, in slot order (the `index` attribute), and a local rule maps those values to the cell's next value. A patch's interior cells keep all their slots: the map of the patch into the landscape sends the reads of each interior cell, in order, to the reads of its image. Then one step of the landscape, read on a patch's interior, is the patch's own step on the restricted configuration (halo exchange), and the steps of patches whose interiors cover the landscape determine the landscape's step.

```
namespace CategoricalCellularAutomataProofs

open CategoryTheory Limits

universe u
```

```
section Limit

variable {J : Type u} [SmallCategory J] {F : J ⥤ Type u} {c : Cocone F}

/-- Configurations of the glued space (functions from its cells to `A`) are the compatible
families of configurations of the patches: a cocone over the patches' cells with apex `A`. -/
def gluedConfigEquiv (hc : IsColimit c) (A : Type u) :
    (c.pt → A) ≃ (F ⟶ (Functor.const J).obj A) :=
  hc.homEquiv

/-- The family a configuration of the glued space gives is its restriction to each patch. -/
theorem gluedConfigEquiv_app (hc : IsColimit c) (A : Type u) (x : c.pt → A) (j : J) :
    (gluedConfigEquiv hc A x).app j = c.ι.app j ≫ x :=
  rfl

/-- **Unique gluing.** Configurations of the patches that agree along every map of the
diagram (on shared cells) are the restrictions of exactly one configuration of the glued
space. -/
theorem exists_unique_glued_config (hc : IsColimit c) {A : Type u} (xs : ∀ j, F.obj j → A)
    (compat : ∀ {j k : J} (f : j → k), F.map f ≫ xs k = xs j) :
    ∃! x : c.pt → A, ∀ j, c.ι.app j ≫ x = xs j := by
  let s : Cocone F :=
    { pt := A
      ι := { app := xs
             naturality := fun j k f ⇒ by simpa using compat f } }
  exact hc.existsUnique s

/-- Two configurations of the glued space that agree on every patch are equal. -/
theorem glued_config_ext (hc : IsColimit c) {A : Type u} {x y : c.pt → A}
    (h : ∀ j, c.ι.app j ≫ x = c.ι.app j ≫ y) : x = y :=
  hc.hom_ext h

end Limit

section Step

variable {X P Q A : Type*}

/-- One synchronous step on a space whose cell `c` reads the cells `nbrs c` in slot order:
the local rule `μ c` maps their values to the next value of `c`. -/
def step (nbrs : X → List X) (μ : X → List A → A) (x : X → A) : X → A :=
  fun c ⇒ μ c ((nbrs c).map x)

/-- A patch of the landscape: the patch's cells `P` with a map `f` into the landscape's
cells, and its interior cells `Q`, included by `ι`, each reading `nbrsP` in slot order. The
map is a map of spaces on the interior: it sends the reads of an interior cell, in order, to
the reads of its image. -/
structure Patch (X P Q : Type*) (nbrs : X → List X) where
  /-- the patch's cells in the landscape -/
  f : P → X
  /-- the interior cells among the patch's cells -/
  ι : Q → P
  /-- the reads of each interior cell, in slot order -/
  nbrsP : Q → List P
  /-- reads are preserved, in order -/
  map_nbrs : ∀ q, (nbrsP q).map f = nbrs (f (ι q))

/-- The patch's own step on its interior, from a configuration of the patch's cells (interior
and halo); the rule of an interior cell is the rule of its image. -/
def Patch.step {nbrs : X → List X} (p : Patch X P Q nbrs) (μ : X → List A → A) (z : P → A) :
    Q → A :=
  fun q ⇒ μ (p.f (p.ι q)) ((p.nbrsP q).map z)

/-- **Halo exchange.** One step of the landscape, read on a patch's interior, is the patch's
own step on the configuration restricted to the patch. -/
```

```
theorem step_restrict {nbrs : X → List X} (p : Patch X P Q nbrs) (μ : X → List A → A)
    (x : X → A) (q : Q) :
    step nbrs μ x (p.f (p.ι q)) = p.step μ (x ∘ p.f) q := by
  simp only [step, Patch.step, ← p.map_nbrs, List.map_map]

/-- **Locality.** The landscape's next values on a patch's interior depend only on the
configuration on the patch. -/
theorem step_local {nbrs : X → List X} (p : Patch X P Q nbrs) (μ : X → List A → A)
    {x y : X → A} (h : x ∘ p.f = y ∘ p.f) (q : Q) :
    step nbrs μ x (p.f (p.ι q)) = step nbrs μ y (p.f (p.ι q)) := by
  rw [step_restrict, step_restrict, h]

/-- **The patches determine the step.** If the interiors of a family of patches cover the
landscape, the landscape's step is the unique configuration whose value on each patch's
interior is that patch's step on the restriction. -/
theorem step_eq_of_cover {ι' : Type*} {P Q : ι' → Type*} {nbrs : X → List X}
    (ps : ∀ k, Patch X (P k) (Q k) nbrs) (μ : X → List A → A)
    (cover : ∀ c, ∃ k q, (ps k).f ((ps k).ι q) = c) (x y : X → A)
    (hy : ∀ k q, y ((ps k).f ((ps k).ι q)) = (ps k).step μ (x ∘ (ps k).f) q) :
    y = step nbrs μ x := by
  funext c
  obtain ⟨k, q, rfl⟩ := cover c
  rw [hy, step_restrict]

end Step

end CategoricalCellularAutomataProofs
```